Milestone 0: The Workspace and Why It Is Shaped This Way

holdem-cfr walkthrough

What this milestone delivers

Milestone 0 is plumbing, but it sets up the one structural decision that matters for the next six weeks. The repository is a Cargo workspace with four crates: `engine`, `games`, `eval` and `cli`. All four share a toolchain pin, a lockfile and a CI pipeline. The pipeline refuses to merge anything that is badly formatted, raises a clippy warning, or fails a test. The pass condition for the milestone was simply that this pipeline goes green, and every later milestone adds tests that run through it automatically.

Crates as an architecture you cannot cheat on

In Rust, a *crate* is the unit of compilation. It is a library or binary with its own `Cargo.toml` that lists exactly which other crates it may use. A *workspace* is a group of crates built together. They share one `Cargo.lock`, so every crate sees identical dependency versions, and one `target/` directory, so shared code is compiled once.

The reason to split the project this way is that dependency direction becomes something the compiler enforces rather than something you have to remember. The `engine` crate depends on nothing. It defines the `Game` trait and the algorithms that consume it: CFR, CFR+, Discounted CFR and best response. Because `engine` cannot even name the `games` crate, it is physically impossible for a poker-specific assumption to leak into the solver. If Kuhn poker, Leduc hold'em and eventually limit hold'em all run through the same engine code untouched, that is evidence the abstraction is genuine. The evidence comes from the build graph, not from a code review.

The `eval` crate, which will hold the 7-card hand evaluator, also stands alone, because ranking poker hands has nothing to do with regret. The `games` crate depends on both: hold'em needs to rank hands, and every game needs the trait. At the top, `cli` depends on everything and produces the `holdem-cfr` binary.

```
          cli  (binary: holdem-cfr)
        /  |  \
   games   |   \
   /    \  |    \
engine   eval    (engine and eval know nothing about each other)
```

Workspace inheritance

The root `Cargo.toml` declares shared metadata once under `[workspace.package]` and shared dependency versions under `[workspace.dependencies]`. Each crate then says `version.workspace = true` or `proptest.workspace = true` instead of repeating values. This is the Rust equivalent of a single source of truth. When we bump `clap` or `proptest`, we change one line.

```
[workspace]
resolver = "3"
members = ["crates/*"]

[workspace.package]
version = "0.1.0"
edition = "2024"
publish = false

[workspace.dependencies]
engine = { path = "crates/engine" }
games = { path = "crates/games" }
eval = { path = "crates/eval" }
clap = { version = "4", features = ["derive"] }
proptest = "1"
```

The `edition = "2024"` line selects the current Rust language edition. Editions are opt-in snapshots of syntax and lint defaults, and every crate in the workspace inherits the same one. `resolver = "3"` is the matching dependency-resolution algorithm. `publish = false` makes it impossible to accidentally push these crates to crates.io under generic names like `engine`.

Build profiles

Cargo builds code under named *profiles*. `dev` is unoptimized and fast to compile. `release` is optimized and slow to compile. `test` is what `cargo test` uses, and by default it inherits from `dev`. That default is a trap for a solver project, because a CFR test on Leduc runs tens of thousands of tree traversals. Unoptimized Rust can easily be thirty to fifty times slower than optimized Rust in tight numeric loops, which turns a half-second test into a coffee break. So the workspace overrides the test profile:

```
[profile.test]
opt-level = 3

[profile.dev.package."*"]
opt-level = 3

[profile.release]
codegen-units = 1
lto = "thin"
debug = "line-tables-only"
```

The second block optimizes third-party dependencies even in dev builds. That is cheap, because they compile once and are cached. The release profile trades compile

time for speed. `codegen-units = 1` lets LLVM see a whole crate at once, `lto = "thin"` lets it inline across crate boundaries (for example, engine calls into games), and the line tables keep stack traces and profilers readable without bloating the binary.

The toolchain pin and CI

`rust-toolchain.toml` tells `rustup` which compiler channel and components to use whenever `cargo` runs inside the repository. Your Mac, GitHub's runner and any future machine therefore build with the same compiler.

The GitHub Actions workflow has three gates, in increasing order of cost. `cargo fmt --check` fails if any file is not in canonical format, which removes style debates permanently. `cargo clippy -- -D warnings` runs Rust's linter and promotes every warning to an error. Clippy catches real bugs as well as style issues, such as needless clones, suspicious comparisons and integer casts that truncate. `cargo test` runs the unit tests, the integration tests in each crate's `tests/` directory, and the property tests. `Swatinem/rust-cache` caches compiled dependencies between runs, so CI takes seconds instead of minutes. It runs on Ubuntu because macOS runners consume a private repo's free Actions minutes about ten times faster.

Git identity

Commits are authored with the GitHub noreply address, set per-repository so it cannot leak into other projects:

```
git config user.name "Tave La Rocque"
git config user.email "269324844+tavelarocque@users.noreply.github.com"
```

Where this leaves us

With the skeleton in place, every later milestone follows the same pattern: write the code, write the test that proves it, and let CI hold the line. Milestone 1 fills in the engine.